AI-Assisted SOC — Operational Runbook

OpenObserve → authenticated webhook → Python receiver → evidence verification → local Ollama → analyst-reviewed SOC report

Version 1.0 | 03 November 2026 | DEPI Project 5 / Week 2

Document control and evidence boundary

Purpose: enable a new operator to reconstruct the existing Windows lab from an authorized copy of its files, start the services, inspect the five configured detection workflows, receive alerts, and locate/review the SOC outputs. This is an operational guide, not a claim of production readiness.

Evidence basis: existing project test history and the OpenObserve configuration screenshots supplied on 03 Nov 2026. Commands labeled VERIFIED were observed previously; commands labeled CHECK are safe inspection steps; commands labeled CONDITIONAL require prerequisites and/or explicit operator authorization. No new end-to-end test is claimed here.

1. Executive operational status

Component	Status	Evidence / caveat
OpenObserve	Configured / historically observed	Native Windows EXE; default org; eight streams and five scheduled alert rules previously observed.
Historical detections	5 / 5 documented	Web burst, SSH brute force, malware behavior, XSS, Exchange export.
AI/SOC reports	5 / 5 artifacts exist	Human analyst validation remains pending; AI conclusions are provisional.
Port 8765 receiver	Partial / startup blocked	SIEM06 and SIEM07 --check fail on 7-day SSH freshness check; SIEM05 check previously passed.
Port 8766 Exchange	Check passed; listener observed	Exchange evidence check passed; TCP listener observed at 127.0.0.1:8766; no new delivery proven.
Live end-to-end acceptance	NOT VERIFIED	No newly confirmed firing→delivery→analysis→report chain during final operational audit.
Week 3 / Week 4	Not started	SOAR containment and final IR phase outside this Week-2

deliverable.

2. System prerequisites and file inventory

Host: Windows with PowerShell, Python 3.10+ and the required Python packages already installed for the project, local Ollama, OpenObserve Windows binary, and authorized copies of the OpenObserve data directory and Week-2 project tree. Internet access is not required for the already-downloaded local model, but software installation/model retrieval may require it.

Known reference layout (adapt drive and root paths for another workstation):

```
D:\OpenObserve-Lab\openobserve.exe
D:\OpenObserve-Lab\data
D:\Desktop\w\Week-2\Integrations\SIEM-OpenObserve
D:\Desktop\w\Week-2\Evidence\SIEM-OpenObserve
D:\Desktop\w\Week-2\Results
```

A screenshot cannot reconstruct the underlying stream data, alert definitions, destination templates, receiver Python source, credentials, or Ollama model weights. Transfer those authorized assets separately. Do not copy only the report files and assume the lab is runnable.

3. Clean-machine setup and portability

1) Copy the complete Week-2 folder and the OpenObserve-Lab directory (including its data) from the approved lab backup while services are stopped. Retain the original as an immutable backup. Verify the transferred file count and hashes if available. 2) Install a compatible Python runtime, Ollama and OpenObserve binary. 3) Ensure the Python dependencies referenced by receiver imports are available; the exact requirements file and package versions have not been verified in the supplied evidence. 4) Review receiver configuration for hard-coded paths, SIEM URL, organization, API credentials, model name and listening ports. 5) Configure a new local webhook secret; never copy secrets from screenshots into documentation. 6) Validate data and alert definitions before any replay.

4. Start OpenObserve (VERIFIED lab command)

```
Set-Location "D:\OpenObserve-Lab"
$env:ZO_DATA_DIR = "D:\OpenObserve-Lab\data"
$env:ZO_SSRF_ALLOW_LOOPBACK = "true"
.\openobserve.exe
```

Keep this PowerShell window open. In a browser open http://localhost:5080 and sign in with credentials provisioned for that machine. Select organization “default”. Loopback allowance is needed for this localhost webhook lab; do not extend this configuration to untrusted production networks.

5. Verify the existing SIEM dataset (CHECK)

In OpenObserve: Data → Streams: confirm the expected eight streams (full inventory must be verified on the target machine). The streams directly evidenced by the five alerts are depi_web_access, depi_auth_logs, depi_malware_sysmon, depi_web_xss, and depi_exchange_events. Use Logs to inspect representative records and the actual timestamp range. Do not re-ingest or rewrite historical timestamps simply to make alerts fire.

Important: OpenObserve “Past 1 Week” is a query-time filter. The SSH stream had 2,000 records visible over “Past 90 Days” but none in “Past 1 Week” during the final check; this blocks some receiver startup validation. A 48-hour alert evaluation window can likewise return zero even though historical events exist.

6. Detection and destination routing

ID	Alert rule	Stream	Destination	Receiver
001	SIEM_WEB_001_HIGH_RATE_HTTP_REQUESTS	depi_web_access	DEPI_SOC_Webhook	8765 /depi-alerts
002	SIEM_AUTH_002_SSH_BRUTE_FORCE	depi_auth_logs	DEPI_SOC_Webhook (verify mapping)	8765 /depi-alerts
003	SIEM_MAL_003_SUSPICIOUS_MALWARE_BEHAVIOR	depi_malware_sysmon	DEPI_SOC_MALWARE_TEST_DEST	8765 /depi-alerts
004	SIEM_WEB_004_XSS_ATTACK_DETECTED	depi_web_xss	DEPI_SOC_XSS_Webhook	8765 /depi-alerts
005	SIEM_EXCHANGE_005_SUSPICIOUS_MAILBOX_EXPORT	depi_exchange_events	DEPI_SOC_EXCHANGE_005_Webhook	8766 /depi-exchange-alerts

Four destination records were visible: DEPI_SOC_EXCHANGE_005_Webhook, DEPI_SOC_MALWARE_TEST_DEST, DEPI_SOC_Webhook, DEPI_SOC_XSS_Webhook. “Used by” counts shown were 1, 1, 2, 1 respectively. The SSH-specific mapping above is inferred from the shared destination count and must be confirmed in its alert rule, not assumed.

7. Webhook configuration (screenshots verified)

OpenObserve → Alerts → Notification Destinations → select the destination → inspect the configured URL, POST method, template and headers. Three destinations point to http://127.0.0.1:8765/depi-alerts; Exchange points to http://127.0.0.1:8766/depi-exchange-alerts. Headers shown: Content-Type: application/json and X-DEPI-Webhook-Key: [REDACTED]. A key was visibly exposed in the supplied screenshots: rotate it before sharing the lab or guide. Do not publish the key or commit it to source control.

Templates: DEPI_SOC_Structured and DEPI_SOC_MALWARE_TEST use raw JSON with alert_name, stream_name and a string-valued rows variable; DEPI_SOC_XSS_Structured uses rows: "{rows:3}". The Exchange template DEPI_SOC_EXCHANGE_005_Structured uses a different Slack-like JSON shape with title/body/fields plus a rows configuration (enabled=true, max=5), and chart disabled. Receiver compatibility must be tested rather than inferred from the template names. The use of quoted {rows} may produce a string, not a native JSON array; receivers must parse/validate this explicitly.

Screenshots show “Skip TLS Verify” enabled on destinations, but all displayed URLs use HTTP loopback; TLS verification is not applicable to those HTTP requests. Do not treat this toggle as a security control. Bind the receivers to 127.0.0.1, and protect the webhook key.

8. XSS alert configuration and zero-row preview

Rule: SIEM_WEB_004_XSS_ATTACK_DETECTED, stream depi_web_xss, Scheduled, checks every 10 minutes, Simple alert, critical when count ≥ 1, cooldown one hour, monitors the last two days (48 hours). Its SQL selects _timestamp, transaction_id, source_ip, request_uri, request_body, response_status, dataset_label, original_event_time and replay_mode; filters include case-normalized script / encoded script / event-handler / javascript patterns in request_uri or request_body. The preview read “WOULD NOT TRIGGER — 0 rows found”. This is consistent with a time-window mismatch and does not prove the historical XSS evidence is missing or the SQL is correct for all encodings.

Do not click Save, Test, or modify the SQL merely to obtain a green preview during a read-only audit. A controlled test would need an authorized event with an in-window _timestamp and documented expected matching behavior.

9. Start Ollama and validate the model (CHECK)

```
ollama list
ollama ps
# Only if the Ollama service is not already running:
ollama serve
```

The previously installed model list included mistral:7b-instruct-v0.3-q4_K_S (project inference model), qwen2.5:7b-instruct-q4_K_M, phi3:mini, llama3.2:3b. “ollama list” confirms installed weights, not service health or a successful inference. Keep “ollama serve” in a dedicated terminal if needed; avoid starting a second server when one is already running.

10. Receiver selection: never compete for port 8765

The following Python entry points are known in the project integration folder. Use ONE port-8765 receiver at a time. Exchange is independent on 8766. The exact CLI support is script-specific; do not assume --help works on every receiver.

```
Set-Location "D:\Desktop\w\Week-2\Integrations\SIEM-OpenObserve"
python -B .\SIEM05_Auto_Triage_Receiver.py --check
python -B .\SIEM06_Multi_Alert_Receiver.py --self-test
python -B .\SIEM06_Multi_Alert_Receiver.py --check
```

```
python -B .\SIEM07_XSS_Receiver_Candidate.py --check
python -B .\SIEM_EXCHANGE_005_AI_Receiver.py --check
```

Observed results: SIEM05 --check previously passed. SIEM06 --self-test passed local contract tests without SIEM writes or AI calls. SIEM06 --check and SIEM07 --check both failed “No recent SSH log sample found in the auth stream”. This is a known 7-day SSH freshness dependency, not evidence of lost SSH records. Exchange --check passed with 3 retrieved records, 2 unique suspicious exports and 1 duplicate removed. A check PASS does not demonstrate new webhook delivery.

11. Launch a receiver (CONDITIONAL; do not run during read-only audit)

```
# Example: Exchange receiver (dedicated PowerShell window)
Set-Location "D:\Desktop\w\Week-2\Integrations\SIEM-OpenObserve"
python -B .\SIEM_EXCHANGE_005_AI_Receiver.py

# Example: SIEM05 (separate window; only if port 8765 is free)
python -B .\SIEM05_Auto_Triage_Receiver.py
```

Exchange receiver previously printed “STATUS: EXCHANGE 005 RECEIVER LISTENING http://127.0.0.1:8766/depi-exchange-alerts” and Windows showed 127.0.0.1:8766 in LISTEN state, PID 25544 at that moment. PID is ephemeral. The Exchange script unexpectedly starts its listener when invoked with --help; avoid using --help for passive inspection. Do not start SIEM06/SIEM07 until the SSH freshness blocker is resolved through an approved change.

12. Safe network checks (CHECK)

```
Get-NetTCPConnection -LocalPort 5080 -State Listen -ErrorAction SilentlyContinue | Select-Object
LocalAddress,LocalPort,State,OwningProcess
Get-NetTCPConnection -LocalPort 8765 -State Listen -ErrorAction SilentlyContinue | Select-Object
LocalAddress,LocalPort,State,OwningProcess
Get-NetTCPConnection -LocalPort 8766 -State Listen -ErrorAction SilentlyContinue | Select-Object
LocalAddress,LocalPort,State,OwningProcess
# Substitute the current PID returned by the preceding check:
Get-CimInstance Win32_Process -Filter "ProcessId = 25544" | Select-Object ProcessId,Name,CommandLine
```

A LISTEN socket proves a process bound the TCP port. Confirm its command line to identify the receiver; it does NOT prove an OpenObserve POST was delivered, authenticated, accepted, processed by AI or written to a SOC report.

13. Alert reception: the actual flow

1. A scheduled alert evaluates a SQL query over its configured lookback and threshold. An alert with zero matching rows will not fire.

2. On a firing event, OpenObserve renders the assigned destination template and issues an HTTP POST to the destination URL. The X-DEPI-Webhook-Key header is the shared-secret authentication mechanism in this lab.
3. The Python listener checks authentication and payload contract. Prior testing saw HTTP 422 invalid_payload errors for malformed requests; an HTTP response alone does not prove correct evidence selection.
4. The receiver correlates the alert with OpenObserve evidence, checks event identity/time windows, deduplicates as applicable, and routes relevant event data to local Mistral.
5. The AI triage output is subjected to validation; qa_status=REVIEW_REQUIRED and semantic warnings require human adjudication. A final SOC report records observed facts, hypotheses, gaps, impact and recommendations without claiming compromise beyond evidence.

14. Controlled acceptance test (NOT YET EXECUTED)

Only after operator approval: capture the pre-test baseline (alert state, stream count, receiver process, log timestamp, output directory contents); use an authorized, clearly labeled lab event with a current _timestamp or an approved replay procedure; confirm the scheduled rule reaches firing; record delivery attempt and HTTP status; confirm receiver accepted the authenticated payload; verify the exact event IDs/source rows; record Ollama invocation and AI output; inspect QA flags; verify a new report and JSON were generated with consistent identifiers; record analyst disposition; then compare before/after file counts and hashes. Never fabricate a successful trigger or overwrite the existing five historical reports.

Acceptance gate: (1) correct source rule/stream; (2) authenticated POST; (3) receiver-specific payload contract; (4) deterministic evidence correlation; (5) model invocation; (6) semantic QA; (7) report creation; (8) human sign-off. A failed gate is a documented blocker, not an excuse to silently relax validation.

15. Five historical SOC cases and exact artifacts

001 — High-rate HTTP

112 requests from 255.102.154.224; threshold 80; provisional Medium; original trigger-to-row link unproven.

AI evidence: Week-2\Evidence\SIEM-OpenObserve\SIEM05_Auto_Alert_AI_Triage_469dc3deb13af6394b.json

SOC report: Week-2\DEPI_P5_SIEM05_SOC_Alert_Report_FINAL_Technical.docx

002 — SSH brute force

30 failed root SSH attempts within one original minute from 183.62.140.253; no successful login evidenced; original year/timezone absent.

AI evidence: Week-2\Evidence\SIEM-OpenObserve\SIEM06_Auth_002_AI_Triage_a823d14b61f061f795.json

SOC report: Week-2\Alert002_SSH_BruteForce_SOC_Report.md

003 — Suspicious malware behavior

One Sysmon process event, ten DNS events and one network event tied by ProcessGuid; suspicious, not proven C2/infection/exfiltration.

AI evidence: Week-2\Evidence\SIEM-OpenObserve\SIEM06_Mal_003_AI_Triage_e05ccfe875e6a53524.json

SOC report: Week-2\Results\SIEM06_MAL_003_FINAL\SIEM06_MAL_003_Final_SOC_Report.md

004 — XSS

XSS-like URI in one transaction, source 192.168.171.129, HTTP 500; no browser execution proven. AI incorrectly referenced HTTP 200; final report corrected it.

AI evidence: Week-2\Evidence\SIEM-OpenObserve\SIEM07_XSS_004_AI_Triage_6ba5792eef667c9cf8.json

SOC report: Week-2\Evidence\SIEM-OpenObserve\SIEM07_XSS_004_Final_SOC_Report.md

005 — Exchange mailbox export

Two unique New-MailboxExportRequest records (1789, 1892) with ASPX output paths; no confirmed web-shell execution or exfiltration.

AI evidence: Week-2\Evidence\SIEM-OpenObserve\SIEM_EXCHANGE_005_21c4515e439a0b3b6a3f_AI_Triage.json

SOC report: Week-2\Results\SIEM_EXCHANGE_005_Final_SOC_Report.md

16. How to review and produce a defensible SOC report

A report is complete only after a reviewer checks: alert name/ID, original versus ingestion timestamps, source stream and log type, raw event references, host/user/IP/process/URI, detection logic, IOC confidence and provenance, model and prompt settings where recorded, analyst-observed facts, alternative benign explanations, missing evidence, severity/confidence justification, containment recommendations (recommendations only in Week 2), QA flags, and analyst approval status. Preserve exact source indicators (e.g. XSS URI and Exchange ASPX paths); do not silently normalize them.

The existing report generation is performed by the receiver/project workflow during historical replay. The provided evidence does not establish a single universal command that regenerates every SOC report from scratch. Therefore, this guide gives verified receiver entry points and exact existing output paths rather than inventing an unsupported “generate all reports” command.

17. Troubleshooting and known defects

SIEM06-TIME-001 — No recent SSH log sample found

SSH evidence exists historically but is outside 7-day startup lookback. Inspect time ranges; propose a reviewed fix, not a silent timestamp rewrite.

SIEM07-STARTUP-001 — Same SSH prerequisite blocks XSS receiver

XSS handling depends on shared preflight. Document dependency and obtain approval before changing code.

EXCHANGE-CLI-001 — --help launches listener

Do not use --help as a read-only query; use source inspection or --check.

XSS-PREVIEW-001 — Would not trigger / 0 rows

48-hour lookback and historical data may not overlap; validate query and timestamps separately.

PAYLOAD-CONTRACT-001 — 422 invalid_payload in earlier tests

Check content type, JSON shape, rows encoding, alert/stream identity and auth without logging secrets.

SEMANTIC-QA-001 — XSS AI said HTTP 200 while raw evidence is 500

Human report correction required; semantic_warnings=[] is not proof of semantic accuracy.

SEMANTIC-QA-002 — Exchange AI omitted/misclassified key evidence

Retain ASPX path indicators, distinguish hypothesis from observation, require human review.

ACCEPTANCE-001 — Listener or --check mistaken for E2E success

Require a newly traceable firing→POST→receiver→AI→report chain.

18. Security, reproducibility and handover checklist

- Rotate the webhook secret exposed in screenshots and update the approved secret store and destination settings together. Never include the secret in handover artifacts.
- Keep OpenObserve and receivers on loopback for this lab; do not expose 5080/8765/8766 to the internet.
- Preserve historical source logs, event hashes and original timestamps. Keep lab replay metadata separate from original event time.
- Copy the full OpenObserve data directory and Week-2 source tree with an inventory/checksum manifest before migrating to another PC.
- Verify the exact Python environment, dependency lockfile (if present), model digest, OpenObserve binary version and alert/destination configuration on the receiving machine.
- Store generated JSONs and SOC reports under the existing paths; avoid duplicate “final” files until acceptance.
- Capture analyst sign-off and a fresh end-to-end acceptance test before declaring Week 2 fully complete.

19. Operator quick start

1. Restore authorized OpenObserve-Lab and Week-2 directories. 2. Start OpenObserve with ZO_DATA_DIR and loopback allowance. 3. Log into localhost:5080/default and verify streams, five alert rules, four destinations and templates. 4. Check timestamps and the 48-hour/7-day window dependencies. 5. Verify Ollama model availability. 6. Run receiver --check/self-test as appropriate. 7.

Launch one port-8765 receiver plus the Exchange receiver on 8766 only when authorized and ready. 8. Verify socket ownership. 9. For historical evidence, review the five listed JSONs and SOC reports. 10. For new reports, execute the controlled acceptance procedure and obtain human validation.

20. Final disposition

WEEK-2 IMPLEMENTATION: DOCUMENTED. FIVE HISTORICAL ALERT WORKFLOWS: EVIDENCED. OPERATIONAL READINESS: PARTIAL. END-TO-END ACCEPTANCE: PENDING. HUMAN ANALYST APPROVAL: PENDING. WEEK-3 SOAR: NOT STARTED.

This distinction protects the integrity of the capstone: successful historical demonstrations, an active listener, and configured webhooks are not equivalent to a fresh, fully traceable, approved end-to-end SOC run.